PRACTICAL - 5

Roll No: 2520030106

Name: D. Mohit Venkat Sai

SEC-8A

Problem Statement :

1. Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

2. Develop a program that executes the equivalent of the shell command **ls -l | grep ".c"** using **fork()**, **pipe()**, **dup2()**, and **exec()** system calls.

Aim :

To implement inter-process communication using an anonymous pipe and measure its communication efficiency, and to reproduce the shell pipeline **ls -l | grep ".c"** using low-level process and pipe system calls.

Procedure :

1. Create an anonymous pipe using **pipe()** and record the start time with **clock_gettime()**.

2. Create a child process using **fork()**.

3. Make the child act as the consumer by reading data from the pipe until the write end is closed.

4. Make the parent act as the producer by generating 100,000 integer values and writing them into the pipe.

5. Wait for the consumer with **waitpid()**, measure elapsed communication time, and calculate throughput in bytes per second.

6. Create a second pipe and two child processes for the command pipeline.

7. Redirect **ls** standard output to the pipe with **dup2()**, execute **ls -l**, redirect the second child's standard input from the pipe, and execute **grep ".c"**.

8. Close unused pipe descriptors and wait for both child processes.

Output :

```
dmvs@DESKTOP-0CVMWSE:/mnt/c/Testing/OSSP_Testing$ gcc Practical_5.c -o Practical_5
./Practical_5

=== Shell Pipeline: ls -l | grep ".c" ===
-rwxrwxrwx 1 dmvs  dmvs       1023 Aug 24 07:09 Practical-3.c
-rwxrwxrwx 1 dmvs  dmvs      16216 Aug 24 06:48 Practical_2
-rwxrwxrwx 1 dmvs  dmvs        715 Aug 24 06:48 Practical_2.c

-rwxrwxrwx 1 dmvs  dmvs      16256 Aug 24 07:10 Practical_3
-rwxrwxrwx 1 dmvs  dmvs      16304 Aug 24 08:11 Practical_4
-rwxrwxrwx 1 dmvs  dmvs      16600 29 12:18 Practical_5
-rwxrwxrwx 1 dmvs  dmvs      16600 24 02:15 Practical_5.c
-rwxrwxrwx 1 dmvs  dmvs      16600 24 07:18 Practical_5
-rwxrwxrwx 1 dmvs  dmvs       3120 29 12:15 Practical_5
-rwxrwxrwx 1 dmvs  dmvs         98 24 06:52 copy.txt
-rwxrwxrwx 1 dmvs  dmvs       1242 24 08:11 practical-4.c

Practical-5 completed successfully.
dmvs@DESKTOP-0CVMWSE:/mnt/c/Testing/OSSP_Testing$
```

The producer-consumer test successfully transferred **400000 bytes**. The measured communication time was **0.060989 seconds**, giving a communication throughput of **6558555.87 bytes/second**.

The pipeline portion successfully executed the equivalent of **ls -l | grep ".c"** and displayed the matching C-related files. The program then reported **Practical-5 completed successfully.**

Source Code - Practical_5.c (1/5):

OSSP_Testing > C Practical_5.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>
#include <time.h>

#define BUFFER_SIZE 4096
#define MESSAGE_COUNT 100000

void producer_consumer()
{
    int fd[2];
    pid_t pid;
    char buffer[BUFFER_SIZE];
    int data = 0;
    ssize_t bytes_read;
    long long total_bytes = 0;
    struct timespec start, end;

    if (pipe(fd) == -1)
    {
        perror("pipe");
        exit(1);
    }

    clock_gettime(CLOCK_MONOTONIC, &start);

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        exit(1);
    }

    if (pid == 0)
```

Source Code - Practical_5.c (2/5):

```
38      {
39          close(fd[1]);
40
41          while ((bytes_read = read(fd[0], buffer, sizeof(buffer))) > 0)
42          {
43              total_bytes += bytes_read;
44          }
45
46          close(fd[0]);
47
48          printf("Consumer process completed.\n");
49          printf("Total bytes consumed: %lld\n", total_bytes);
50
51          exit(0);
52      }
53      else
54      {
55          close(fd[0]);
56
57          for (int i = 0; i < MESSAGE_COUNT; i++)
58          {
59              data = i + 1;
60
61              if (write(fd[1], &data, sizeof(data)) == -1)
62              {
63                  perror("write");
64                  close(fd[1]);
65                  exit(1);
66              }
67          }
68
69          close(fd[1]);
70
71          waitpid(pid, NULL, 0);
72
73          clock_gettime(CLOCK_MONOTONIC, &end);
```

Source Code - Practical_5.c (3/5):

```
75          double elapsed =
76              (end.tv_sec - start.tv_sec) +
77              (end.tv_nsec - start.tv_nsec) / 1000000000.0;
78
79          long long total_bytes_sent =
80              (long long)MESSAGE_COUNT * sizeof(int);
81
82          double throughput =
83              total_bytes_sent / elapsed;
84
85          printf("Producer process completed.\n");
86          printf("Total bytes produced: %lld\n", total_bytes_sent);
87          printf("Communication time: %.6f seconds\n", elapsed);
88          printf("Communication throughput: %.2f bytes/second\n", throughput);
89      }
90  }
91
92  void execute_pipeline()
93  {
94      int fd[2];
95      pid_t p1, p2;
96
97      if (pipe(fd) == -1)
98      {
99          perror("pipe");
100         exit(1);
101     }
102
103     p1 = fork();
104
105     if (p1 < 0)
106     {
107         perror("fork");
108         exit(1);
109     }
```

Source Code - Practical_5.c (4/5):

```
110
111     if (p1 == 0)
112     {
113         close(fd[0]);
114
115         dup2(fd[1], STDOUT_FILENO);
116         close(fd[1]);
117
118         execlp("ls", "ls", "-l", (char *)NULL);
119
120         perror("execlp ls");
121         exit(1);
122     }
123
124     p2 = fork();
125
126     if (p2 < 0)
127     {
128         perror("fork");
129         exit(1);
130     }
131
132     if (p2 == 0)
133     {
134         close(fd[1]);
135
136         dup2(fd[0], STDIN_FILENO);
137         close(fd[0]);
138
139         execlp("grep", "grep", ".c", (char *)NULL);
140
141         perror("execlp grep");
142         exit(1);
143     }
144
145     close(fd[0]);
```

Source Code - Practical_5.c (5/5):

```c
    close(fd[0]);
    close(fd[1]);

    waitpid(p1, NULL, 0);
    waitpid(p2, NULL, 0);
}

int main()
{
    printf("=== Producer-Consumer Communication ===\n");

    producer_consumer();

    printf("\n=== Shell Pipeline: ls -l | grep \".c\" ===\n");

    execute_pipeline();

    printf("\nPractical-5 completed successfully.\n");

    return 0;
}
```

Observation and Analysis

Producer-Consumer Communication

The parent process functions as the producer and writes 100,000 integer values to the anonymous pipe. Since each integer occupies 4 bytes, the total transmitted data is 400,000 bytes. The child process acts as the consumer and reads from the pipe until the producer closes its write descriptor.

The measured communication time is 0.060989 seconds. Using the program's calculation, the resulting communication throughput is approximately 6.56 MB/s (6,558,555.87 bytes/s).

Shell Pipeline Implementation

The second part constructs the pipeline using two child processes and an anonymous pipe. The first child closes the read end, connects the pipe's write end to standard output using **dup2()**, and executes **ls -l** with **execlp()**. The second child connects the pipe's read end to standard input and executes **grep ".c"**.

The observed output contains entries such as **Practical-3.c**, **Practical_2.c**, **Practical_5.c**, and **practical-4.c**, demonstrating that the pipeline filters the directory listing for matching C files.

Result :

Both requirements of Practical-5 were successfully implemented in C. Anonymous pipes were used for producer-consumer communication and the measured throughput was **6558555.87 bytes/second**. The shell command **ls -l | grep ".c"** was also successfully reproduced using **fork()**, **pipe()**, **dup2()**, and **execlp()**.